Assignment 2

DevOps Internship

Docker & Kubernetes

Q1:

What is *Docker*? Explain any five benefits of using Docker in a DevOps workflow.

Docker is an open-source platform for packaging and running applications in lightweight containers. A container includes the application code, dependencies, and configuration needed to run consistently in different environments.

Five benefits of Docker in DevOps are:

- It isolates applications and avoids dependency conflicts.
- It uses fewer resources than virtual machines.
- It makes applications easier to scale and manage.
- It enables faster application deployment.
- It provides consistency across development, testing, and production.

Q2:

What is *containerization*? Explain how it differs from traditional application deployment.

Containerization is the process of packaging an application, along with its dependencies and configuration files, into a lightweight container.

Traditional application deployment installs applications directly on a server or virtual machine, which can cause dependency conflicts and require more resources. Containers are isolated, portable, start faster, and run consistently across different environments.

Q3:

Explain the difference between a *Virtual Machine* and a *Docker Container* with at least five points.

Virtual Machine	Docker Container
Runs a complete operating system.	Shares the host operating system kernel.
Uses more CPU, memory, and storage.	Uses fewer system resources.
Takes longer to start.	Starts within seconds.
Provides hardware-level isolation.	Provides process-level isolation.
Has a larger image size.	Has a smaller image size.
Is slower to deploy and scale.	Is faster to deploy and scale.

Q4:

Explain the Docker architecture and describe the role of the following components:

Docker uses a client-server architecture. The Docker Client sends commands to the Docker Daemon, which builds, runs, and manages containers.

Docker Client:

It is a tool that allows users to give Docker instructions. It sends commands such as *docker build*, *docker pull*, and *docker run* to the Docker Daemon for execution.

Docker Daemon:

A background service that receives commands from the Docker Client and performs tasks such as creating images, starting containers, and managing Docker resources.

Docker Image:

It is a read-only template containing the application code, dependencies, and configuration.

Docker Container:

It is a running instance of a Docker image.

Docker Registry:

It stores and distributes Docker images. Docker Hub is a commonly used registry.

Q5:

Write the Docker commands to perform the following tasks:

- **Pull the nginx image:**

  docker pull nginx

- **List all images:**

  docker images

- **Run a container from the nginx image**

  docker run -d --name container-name nginx

- **List all running containers**

  docker ps

- **Stop and remove the container**

  docker stop container-name

  docker rm container-name

  docker rm -f container-name (This command stops and removes a container at the same time)

Q6:

What is a *Docker Image*? Explain the difference between an image and a container with a suitable example.

A **Docker image** is a *read-only template* that contains application code, dependencies, libraries, and configuration required to run an application.

An image is the blueprint, while a **container** is a running instance of that image. One image can be used to create multiple containers.

For example, the “nginx” image contains everything needed to run an Nginx web server. When we run “docker run nginx”, Docker creates and starts a container from that image.

Q7:

What is a *Dockerfile*? Write a simple Dockerfile that runs a basic Python or Node.js application.

A ***Dockerfile*** is a text file containing instructions for building a Docker image.

Example Dockerfile for a basic Python application:

FROM python:3.12-slim

WORKDIR /app

COPY app.py .

CMD ["python", "app.py"]

Here, “FROM” selects the Python base image, “WORKDIR” sets the application directory, “COPY” adds the Python file, and “CMD” runs the application.

Q8:

Explain the purpose of the following Dockerfile instructions with one example each:

- **FROM**

  It specifies the base image used to build the Docker image.

  Example: FROM python:3.12-slim

- **WORKDIR**

  It sets the default directory inside the Docker image where the following commands will run. If the directory does not exist, Docker creates it automatically.

  Example: WORKDIR /app

  After this instruction, commands such as COPY, RUN, and CMD will use /app as the current directory.

  COPY

  It copies files or folders from the local project directory into the Docker image. It is commonly used to add application code and configuration files.

  Example: COPY app.py /app/

  This copies app.py from the local computer into the /app directory inside the image.

- **RUN**

  It executes a command while the Docker image is being built. It is commonly used to install packages, dependencies, or system tools.

  Example: RUN pip install flask

  This installs Flask inside the Docker image so that it is available when the container starts.

- **CMD**

  It specifies the default command to run when the container starts.

  Example: CMD ["python", "app.py"]

Q9:

What are *Docker Volumes*? Explain why they are needed and write the command to create and mount a volume.

Docker volumes are used to save data created by a container. They are needed because data inside a container may be lost when the container is deleted. A volume keeps the data safe and can also be used by more than one container.

1- Create a volume:

docker volume create my-volume

2- Run a container and mount the volume:

docker run -d --name nginx-container -v my-volume:/usr/share/nginx/html nginx

In this command, “my-volume” stores the data from the “/usr/share/nginx/html” folder inside the container.

Q10:

Research three Docker commands that were not covered during the training and explain their purpose with an example for each.

1- pause

It temporarily pauses all processes running inside a container.

Example: docker pause nginx-container

2- system df

It shows how much disk space is being used by Docker images, containers, and volumes.

Example: docker system df

3- top

It displays the processes running inside a container.

Example: docker top nginx-container

Q11:

What is *Kubernetes*? Explain any five reasons why organizations use Kubernetes to manage containers.

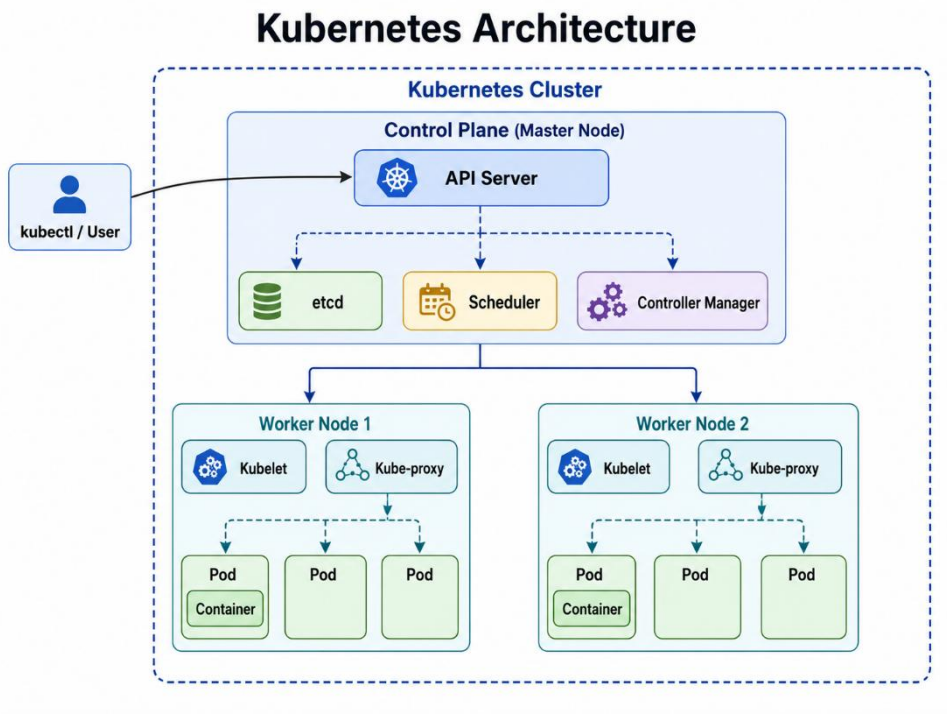
Kubernetes is an open-source platform for deploying, managing, and scaling containerized applications.

The following are five reasons why organizations use Kubernetes to manage containers:

- It automatically starts, stops, and manages containers.
- It can increase or decrease the number of containers based on demand.
- It uses computing resources efficiently by placing containers on suitable servers.
- It supports rolling updates and makes it easier to return to an older version.
- It supports easy application updates without causing long downtime.

Q12:

Explain the *Kubernetes architecture* and describe the role of the following components:



Kubernetes uses a control plane and worker-node architecture. The control plane manages the cluster, while worker nodes run the applications.

- **API Server:**

  Receives commands and acts as the main communication point for the Kubernetes cluster.

- **etcd**

  Stores all cluster information, including configuration, resources, and the current state.

- **Scheduler**

  Selects the most suitable worker node for running a new Pod.

- **Kubelet**

  Runs on each worker node and ensures that the required containers are running properly.

- **Kube-proxy**

  Manages network communication and ensures traffic reaches the correct Pods and services.

Q13:

What is a *Pod* in Kubernetes? Write a simple Pod YAML manifest that runs an *nginx* container.

A Pod is the smallest unit in Kubernetes. It contains one or more containers that run together and share the same network and storage.

create-pod.yaml

apiVersion: v1

kind: Pod

metadata:

name: nginx-pod

spec:

containers:

- name: nginx-container

image: nginx

ports:

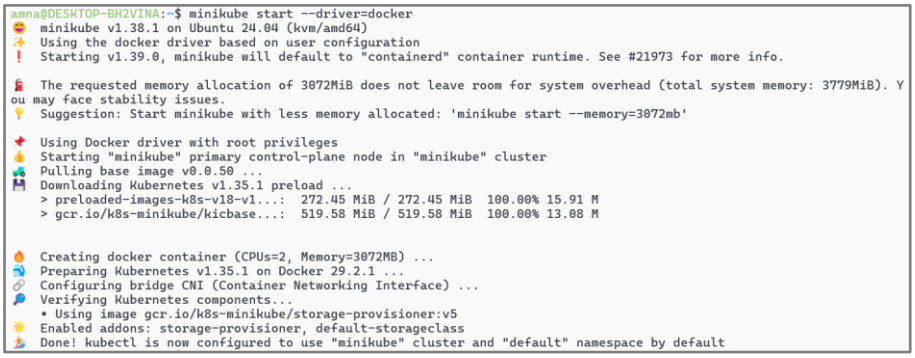
- containerPort: 80

This YAML creates a Pod named nginx-pod and runs an Nginx container on port 80.

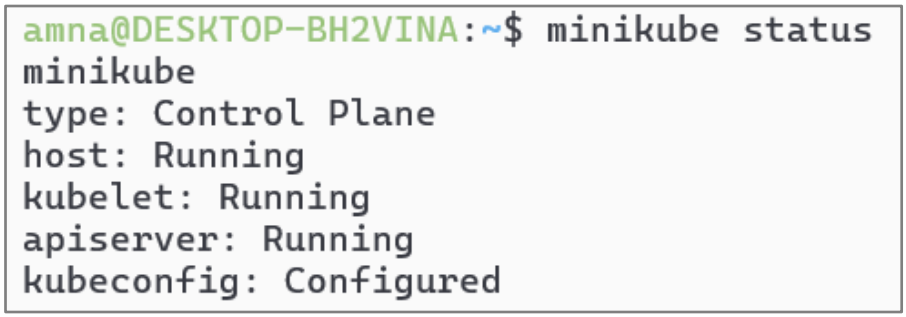
Q14:

Install *Minikube* on your machine and write the commands to:

- **Start the cluster**



- **Check the cluster status**



- **View all nodes**



- **View all pods**

```
amna@DESKTOP-BH2VINA:~$ kubectl get pods
No resources found in default namespace.
amna@DESKTOP-BH2VINA:~$ kubectl get pods --all-namespaces
NAMESPACE     NAME                                 READY   STATUS    RESTARTS   AGE
kube-system   coredns-7d764666f9-zx2t4             1/1     Running   0          4m28s
kube-system   etcd-minikube                        1/1     Running   0          4m33s
kube-system   kube-apiserver-minikube              1/1     Running   0          4m36s
kube-system   kube-controller-manager-minikube     1/1     Running   0          4m33s
kube-system   kube-proxy-5wwbk                     1/1     Running   0          4m28s
kube-system   kube-scheduler-minikube              1/1     Running   0          4m33s
kube-system   storage-provisioner                  1/1     Running   0          4m19s
```

The --all-namespaces option displays resources across the entire cluster rather than only the current namespace.

Q15:

What is a *Deployment* in Kubernetes? Explain how it is different from a Pod.

A Deployment is a Kubernetes object used to create, manage, and update multiple copies of a Pod. It ensures that the required number of Pods are always running.

Difference between a Deployment and a Pod:

Pod	Deployment
A Pod is the smallest unit in Kubernetes.	A Deployment is used to manage Pods.
It usually runs one or more containers.	It creates and controls multiple Pod replicas.
It does not support automatic scaling on its own.	It can increase or decrease the number of Pods.
If a standalone Pod is deleted, it is not recreated automatically.	If a managed Pod fails, the Deployment creates a replacement.
Updating a Pod usually requires creating a new Pod.	A Deployment supports rolling updates and rollbacks.
A Pod is suitable for simple testing.	A Deployment is preferred for running applications in production.

In short, a ***Pod*** runs one or more containers but does not automatically restart if it is deleted. A ***Deployment*** manages Pods, replaces failed Pods, supports scaling, and allows application updates and rollbacks.

Q16:

Explain the three main types of Kubernetes *Services* — ClusterIP, NodePort, and LoadBalancer — with a use case for each.

Kubernetes Services provide a stable way to access Pods.

Service Type	Purpose	Use Case
ClusterIP	Makes the application accessible only inside the Kubernetes cluster.	Internal communication between frontend and backend services.
NodePort	Opens a port on every worker node so the application can be accessed from outside the cluster.	Accessing an application during development or testing.

LoadBalancer	Creates an external load balancer and sends traffic to the Pods.	Hosting a public production website in the cloud.

Q17:

What is a *ConfigMap* and a *Secret*? Explain the difference between them with an example.

A **ConfigMap** stores non-sensitive configuration data, such as application settings, URLs, or port numbers.

A **Secret** stores sensitive data, such as passwords, API keys, and login details.

Difference between ConfigMap and Secrets:

ConfigMap	Secrets
Stores normal configuration data.	Stores sensitive information.
Used for values such as database host or application mode.	Used for values such as passwords or tokens.
Data is usually stored as plain text.	Data is commonly stored in encoded form and can be protected with extra security controls.

Example:

A **ConfigMap** may store:

APP_MODE: production

DATABASE_HOST: mysql-service

A **Secret** may store:

DATABASE_USER: admin

DATABASE_PASSWORD: mypassword

Both can be passed to Pods as environment variables or mounted as files.

Q18:

Research any two Kubernetes commands (kubectl) that were not covered during the training and explain their purpose.

1- **kubectl debug**

This command is used to troubleshoot Pods, containers, and nodes. It can add a temporary debugging container containing tools that may not be available in the original application container.

Example:
kubectl debug nginx-pod -it --image=busybox

This starts a BusyBox debugging container inside the nginx-pod.

2- kubectl auth can-i

This command checks whether a user or service account has permission to perform a specific action in the cluster. It is useful for testing Kubernetes RBAC permissions.

Example:
kubectl auth can-i delete pods -n production

This checks whether the current user is allowed to delete Pods in the production namespace.

Q19:

Explain what *Grafana* is used for and how it helps in monitoring a Kubernetes cluster.

Grafana is an open-source monitoring and visualization tool. It displays system data in dashboards using graphs, charts, and alerts.

In a Kubernetes cluster, Grafana helps monitor CPU usage, memory usage, Pod status, node performance, and application health. It usually collects metrics from tools such as Prometheus. Grafana helps teams quickly find performance problems, track resource usage, and receive alerts when something goes wrong.